

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский национальный исследовательский политехнический
университет»

Электротехнический факультет

Кафедра: Информационные технологии и автоматизированные системы
(ИТАС)

Направление подготовки: 09.03.04 – «Программная инженерия»

ОТЧЕТ

Лабораторная работа №12.

Тема: «»

По дисциплине «Основы алгоритмизации и программирования»

Вариант 10

Выполнила: студентка группы РИС-25-26

Абрамова Валерия Андреевна

Принял: доц. Полякова О.А.

Пермь, 2026

Постановка задачи

Создать ассоциативный контейнер (``multimap``) для элементов типа ``float`` (задача 1) и для пользовательского типа ``Money`` (задача 2). Создать параметризованный класс «Вектор», использующий внутри ассоциативный контейнер (задача 3). Заполнить контейнеры элементами, затем добавить элементы на заданную позицию (минимальный элемент), удалить элементы, большие среднего арифметического, и каждый элемент домножить на максимальный элемент контейнера. Для задач 1–2 использовать глобальные функции, для задачи 3 — методы класса.

Анализ задачи

Задача 1 (`multimap int, float`)

1. Создаём `multimap` с ключом типа `int` и значением типа `float`.
2. Вводим с клавиатуры 5 пар (ключ и значение), добавляем их в `multimap` через `insert`.
3. Выводим все пары на экран.
4. Находим минимальное значение (перебираем все элементы, сравниваем `second`).
5. Запрашиваем у пользователя ключ для вставки минимального элемента.
6. Вставляем новую пару (введённый ключ, минимальное значение) через `insert`.
7. Выводим обновлённый `multimap`.

Задача 2 (`multimap int, Money`)

1. Создаём `multimap` с ключом типа `int` и значением типа `Money`.
2. Вводим с клавиатуры 3 пары (ключ и сумму `Money`), добавляем их в `multimap`.
3. Вычисляем среднее арифметическое всех значений (суммируем `toDouble()`, делим на размер).
4. Перебираем `multimap` и удаляем все элементы, значение которых больше среднего (используем итератор с `erase`, возвращающим следующий итератор).
5. Выводим оставшиеся пары.

Задача 3 (`MyMap int, Money` - удаление больше среднего)

1. Используем шаблонный класс `MyMap`, который внутри хранит `multimap` и предоставляет методы `add`, `print`, `removeGreaterThanAverage`, `findAverage`.
2. Создаём `MyMap` с ключом `int` и значением `Money`, вводим 3 пары через метод `add`.
3. Вызываем метод `removeGreaterThanAverage`, который вычисляет среднее арифметическое через `findAverage` и удаляет элементы со значением больше среднего.
4. Выводим результат через метод `print`.

Задача 4 (`MyMap int, Money` - умножение на максимум)

1. Создаём `MyMap` с ключом `int` и значением `Money`, вводим 3 пары через метод `add`.

2. Находим максимальный элемент через метод `findMax` (перебираем все значения, сравниваем `operator>`).
3. Вызываем метод `multiplyByMax`, который создаёт новый `multimap`, умножает каждый элемент (кроме максимального) на максимальный через `operator*` (переводим в копейки, умножаем, нормализуем), затем заменяет старые данные новыми.
4. Выводим результат через метод `print`.

Задача 5 (`MyMap<int, Money>` - добавление минимума)

1. Создаём `MyMap` с ключом `int` и значением `Money`, вводим 3 пары через метод `add`.
2. Находим минимальный элемент через метод `findMin` (перебираем все значения, сравниваем `operator<`).
3. Запрашиваем у пользователя ключ для вставки минимального элемента.
4. Вызываем метод `addMinToPosition`, который вставляет новую пару (указанный ключ, минимальное значение) через `insert` в `multimap`.
5. Выводим результат через метод `print`.

Код

```
#include <iostream>
#include <map>
#include <locale>
#include <vector>
```

```
using namespace std;
```

```
class Money {
private:
    long rubles;
    int kopeks;
```

```
public:
```

```
    Money() : rubles(0), kopeks(0) {}
```

```
    Money(long r, int k) : rubles(r), kopeks(k) {
        normalize();
    }
```

```
    Money(const Money& other) : rubles(other.rubles), kopeks(other.kopeks) {}
```

```
    ~Money() {}
```

```
    Money& operator=(const Money& other) {
        if (this != &other) {
```

```

        rubles = other.rubles;
        kopeks = other.kopeks;
    }
    return *this;
}

bool operator==(const Money& other) const {
    return (rubles == other.rubles && kopeks == other.kopeks);
}

bool operator!=(const Money& other) const {
    return !(*this == other);
}

bool operator>(const Money& other) const {
    if (rubles != other.rubles) {
        return rubles > other.rubles;
    }
    return kopeks > other.kopeks;
}

bool operator<(const Money& other) const {
    if (rubles != other.rubles) {
        return rubles < other.rubles;
    }
    return kopeks < other.kopeks;
}

bool operator>=(const Money& other) const {
    return (*this > other) || (*this == other);
}

bool operator<=(const Money& other) const {
    return (*this < other) || (*this == other);
}

Money operator+(const Money& other) const {
    Money result;
    result.rubles = rubles + other.rubles;
    result.kopeks = kopeks + other.kopeks;
    result.normalize();
    return result;
}

Money operator*(int multiplier) const {

```

```

Money result;
long total = (rubles * 100 + kopeks) * multiplier;
result.rubles = total / 100;
result.kopeks = total % 100;
if (result.kopeks < 0) {
    result.rubles--;
    result.kopeks += 100;
}
result.normalize();
return result;
}

```

```

Money operator*(const Money& other) const {
    Money result;
    double val1 = rubles + kopeks / 100.0;
    double val2 = other.rubles + other.kopeks / 100.0;
    double product = val1 * val2;
    long total = product * 100;
    result.rubles = total / 100;
    result.kopeks = total % 100;
    if (result.kopeks < 0) {
        result.rubles--;
        result.kopeks += 100;
    }
    result.normalize();
    return result;
}

```

```

Money& operator+=(const Money& other) {
    rubles += other.rubles;
    kopeks += other.kopeks;
    normalize();
    return *this;
}

```

```

Money operator-(const Money& other) const {
    Money result;
    long total1 = rubles * 100 + kopeks;
    long total2 = other.rubles * 100 + other.kopeks;
    long diff = total1 - total2;
    result.rubles = diff / 100;
    result.kopeks = diff % 100;
    if (result.kopeks < 0) {
        result.rubles--;
        result.kopeks += 100;
    }
}

```

```

    }
    return result;
}

```

```

Money operator--(int) {
    Money temp = *this;
    kopeks -= 1;
    normalize();
    return temp;
}

```

```

Money& operator--() {
    kopeks -= 1;
    normalize();
    return *this;
}

```

```

double toDouble() const {
    return rubles + kopeks / 100.0;
}

```

```

void normalize() {
    if (kopeks >= 100) {
        rubles += kopeks / 100;
        kopeks %= 100;
    }
    else if (kopeks < 0) {
        long newRubles = rubles + (kopeks / 100) - 1;
        int newKopeks = 100 + (kopeks % 100);
        if (newKopeks >= 100) {
            newRubles += newKopeks / 100;
            newKopeks %= 100;
        }
        rubles = newRubles;
        kopeks = newKopeks;
    }
}

```

```

friend istream& operator>>(istream& in, Money& m);
friend ostream& operator<<(ostream& out, const Money& m);
};

```

```

istream& operator>>(istream& in, Money& m) {
    cout << "Введите рубли: ";
    in >> m.rubles;
}

```

```

    cout << "Введите копейки: ";
    in >> m.kopeks;
    m.normalize();
    return in;
}

ostream& operator<<(ostream& out, const Money& m) {
    out << m.rubles << ",";
    if (m.kopeks < 10) {
        out << "0";
    }
    out << m.kopeks;
    return out;
}

void task1() {
    cout << "\nЗАДАЧА 1" << endl;
    cout << "Контейнер: multimap\nТип: float\n" << endl;

    multimap<int, float> mmap;

    cout << "Введите 5 пар (ключ int, значение float):" << endl;
    for (int i = 0; i < 5; i++) {
        int key;
        float value;
        cout << "Пара " << i + 1 << " - ключ: ";
        cin >> key;
        cout << "значение: ";
        cin >> value;
        mmap.insert({ key, value });
    }

    cout << "\nИсходный multimap:" << endl;
    for (auto& p : mmap) {
        cout << "Ключ: " << p.first << ", Значение: " << p.second << endl;
    }

    float minVal = mmap.begin()->second;
    for (auto& p : mmap) {
        if (p.second < minVal) {
            minVal = p.second;
        }
    }
    cout << "\nМинимальный элемент: " << minVal << endl;
}

```

```

int pos;
cout << "Введите ключ для вставки минимального элемента: ";
cin >> pos;

mmap.insert({ pos, minVal });

cout << "\nРезультат после добавления:" << endl;
for (auto& p : mmap) {
    cout << "Ключ: " << p.first << ", Значение: " << p.second << endl;
}
}

void task2() {
    cout << "\nЗАДАЧА 2" << endl;
    cout << "Контейнер: multimap\nТип: Money\n" << endl;

    multimap<int, Money> mmap;

    cout << "Введите 3 пары (ключ int, значение Money):" << endl;
    for (int i = 0; i < 3; i++) {
        int key;
        Money value;
        cout << "\nПара " << i + 1 << endl;
        cout << "Ключ: ";
        cin >> key;
        cout << "Значение (Money):" << endl;
        cin >> value;
        mmap.insert({ key, value });
    }

    cout << "\nИсходный multimap:" << endl;
    for (auto& p : mmap) {
        cout << "Ключ: " << p.first << ", Значение: " << p.second << endl;
    }

    double sum = 0;
    for (auto& p : mmap) {
        sum += p.second.toDouble();
    }
    double avg = sum / mmap.size();
    cout << "\nСреднее арифметическое: " << avg << endl;

    cout << "\nЭлементы больше среднего:" << endl;
    for (auto& p : mmap) {
        if (p.second.toDouble() > avg) {

```



```

        cout << "Удален: ключ " << p.first << ", значение " << p.second << endl;
    }
}

for (auto it = mmap.begin(); it != mmap.end(); ) {
    if (it->second.toDouble() > avg) {
        it = mmap.erase(it);
    }
    else {
        ++it;
    }
}

cout << "\nРезультат после удаления:" << endl;
if (mmap.empty()) {
    cout << "Multimap пуст!" << endl;
}
else {
    for (auto& p : mmap) {
        cout << "Ключ: " << p.first << ", Значение: " << p.second << endl;
    }
}
}

template <typename KeyType, typename ValueType>
class MyMap {
private:
    multimap<KeyType, ValueType> data;

public:
    MyMap() {}

    void add(const KeyType& key, const ValueType& value) {
        data.insert({ key, value });
    }

    void remove(const KeyType& key) {
        data.erase(key);
    }

    void remove(const KeyType& key, const ValueType& value) {
        for (auto it = data.begin(); it != data.end(); ) {
            if (it->first == key && it->second == value) {
                it = data.erase(it);
            }
        }
    }
}

```

```

        else {
            ++it;
        }
    }
}

int size() const {
    return data.size();
}

void print() const {
    if (data.empty()) {
        cout << "Контейнер пуст!" << endl;
        return;
    }
    for (auto& p : data) {
        cout << "Ключ: " << p.first << ", Значение: " << p.second << endl;
    }
}

multimap<KeyType, ValueType>& getData() {
    return data;
}

const multimap<KeyType, ValueType>& getData() const {
    return data;
}

ValueType findMin() const {
    if (data.empty()) {
        throw "Контейнер пуст!";
    }
    ValueType minVal = data.begin()->second;
    for (auto& p : data) {
        if (p.second < minVal) {
            minVal = p.second;
        }
    }
    return minVal;
}

double findAverage() const {
    if (data.empty()) return 0;
    double sum = 0;
    for (auto& p : data) {

```

```

        sum += p.second.toDouble();
    }
    return sum / data.size();
}

void removeGreaterThanAverage() {
    if (data.empty()) return;
    double avg = findAverage();
    for (auto it = data.begin(); it != data.end(); ) {
        if (it->second.toDouble() > avg) {
            it = data.erase(it);
        }
        else {
            ++it;
        }
    }
}

ValueType findMax() const {
    if (data.empty()) {
        throw "Контейнер пуст!";
    }
    ValueType maxVal = data.begin()->second;
    for (auto& p : data) {
        if (p.second > maxVal) {
            maxVal = p.second;
        }
    }
    return maxVal;
}

void multiplyByMax() {
    if (data.empty()) return;
    ValueType maxVal = findMax();
    multimap<KeyType, ValueType> newData;
    for (auto& p : data) {
        if (p.second != maxVal) {
            Money product = p.second * maxVal;
            newData.insert({ p.first, product });
        }
        else {
            newData.insert({ p.first, p.second });
        }
    }
    data = newData;
}

```

```

    }

    void addMinToPosition(const KeyType& pos) {
        if (data.empty()) return;
        ValueType minVal = findMin();
        data.insert({ pos, minVal });
    }
};

void task3() {
    cout << "\nЗАДАЧА 3" << endl;
    cout << "Параметризированный класс: MyMap\nТип: int, Money\n" << endl;

    MyMap<int, Money> мумар;

    cout << "Введите 3 пары (ключ int, значение Money):" << endl;
    for (int i = 0; i < 3; i++) {
        int key;
        Money value;
        cout << "\nПара " << i + 1 << endl;
        cout << "Ключ: ";
        cin >> key;
        cout << "Значение (Money):" << endl;
        cin >> value;
        мумар.add(key, value);
    }

    cout << "\nИсходный контейнер:" << endl;
    мумар.print();

    мумар.removeGreaterThanOrAverage();

    cout << "\nРезультат после удаления элементов больше среднего:" << endl;
    if (мумар.size() == 0) {
        cout << "Контейнер пуст!" << endl;
    }
    else {
        мумар.print();
    }
}

void task4() {
    cout << "\nЗАДАЧА 4 (Дополнительно)" << endl;
    cout << "MyMap с операцией умножения на максимальный элемент\n" <<
endl;

```

```
MyMap<int, Money> мумар;
```

```
cout << "Введите 3 пары (ключ int, значение Money):" << endl;
for (int i = 0; i < 3; i++) {
    int key;
    Money value;
    cout << "\nПара " << i + 1 << endl;
    cout << "Ключ: ";
    cin >> key;
    cout << "Значение (Money):" << endl;
    cin >> value;
    мумар.add(key, value);
}
```

```
cout << "\nИсходный контейнер:" << endl;
мумар.print();
```

```
Money maxVal = мумар.findMax();
cout << "\nМаксимальный элемент: " << maxVal << endl;
```

```
мумар.multiplyByMax();
```

```
cout << "\nРезультат после умножения каждого элемента на максимальный:"
<< endl;
мумар.print();
}
```

```
void task5() {
    cout << "\nЗАДАЧА 5 (Дополнительно)" << endl;
    cout << "MyMap с операцией добавления минимального элемента\n" << endl;
```

```
MyMap<int, Money> мумар;
```

```
cout << "Введите 3 пары (ключ int, значение Money):" << endl;
for (int i = 0; i < 3; i++) {
    int key;
    Money value;
    cout << "\nПара " << i + 1 << endl;
    cout << "Ключ: ";
    cin >> key;
    cout << "Значение (Money):" << endl;
    cin >> value;
    мумар.add(key, value);
}
```

```

cout << "\nИсходный контейнер:" << endl;
mymap.print();

Money minVal = mymap.findMin();
cout << "\nМинимальный элемент: " << minVal << endl;

int pos;
cout << "Введите ключ для вставки минимального элемента: ";
cin >> pos;

mymap.addMinToPosition(pos);

cout << "\nРезультат после добавления минимального элемента:" << endl;
mymap.print();
}

int main() {
    setlocale(LC_ALL, "Russian");

    int choice;

    do {
        cout << "\nГЛАВНОЕ МЕНЮ" << endl;
        cout << "1. Задача 1 (multimap<int, float>)" << endl;
        cout << "2. Задача 2 (multimap<int, Money>)" << endl;
        cout << "3. Задача 3 (MyMap<int, Money> - удаление > среднего)" << endl;
        cout << "4. Задача 4 (MyMap<int, Money> - умножение на максимум)" <<
endl;
        cout << "5. Задача 5 (MyMap<int, Money> - добавление минимума)" <<
endl;
        cout << "0. Выход" << endl;
        cout << "Ваш выбор: ";
        cin >> choice;

        switch (choice) {
            case 1:
                task1();
                break;
            case 2:
                task2();
                break;
            case 3:
                task3();
                break;

```

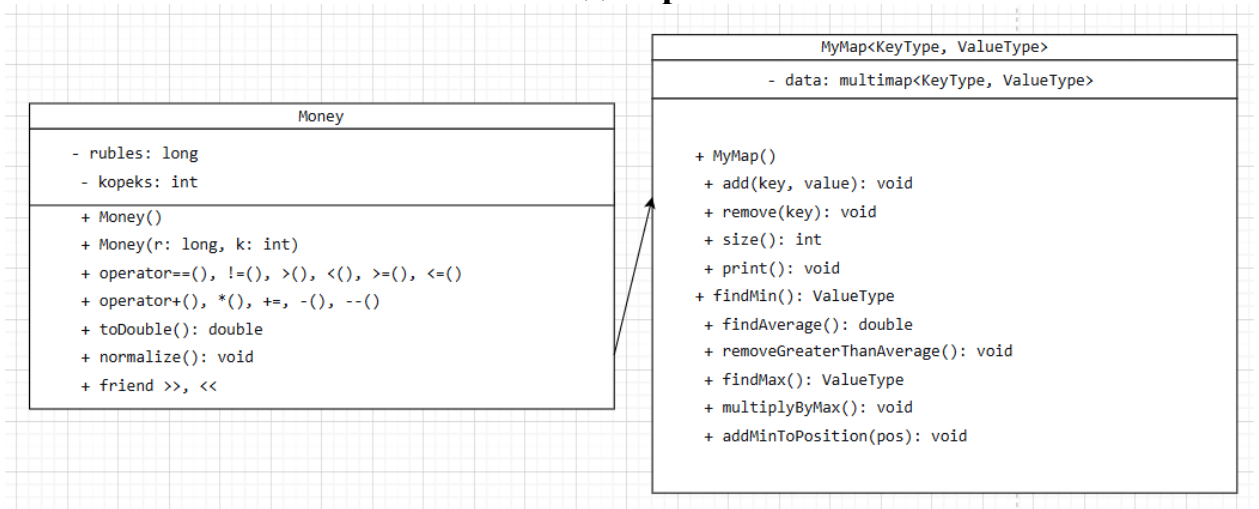
```

case 4:
    task4();
    break;
case 5:
    task5();
    break;
case 0:
    cout << "Выход из программы..." << endl;
    break;
default:
    cout << "Неверный выбор!" << endl;
}
} while (choice != 0);

return 0;
}

```

UML диаграмма



Вывод программы

ГЛАВНОЕ МЕНЮ

1. Задача 1 (multimap<int, float>)
2. Задача 2 (multimap<int, Money>)
3. Задача 3 (MyMap<int, Money> - удаление > среднего)
4. Задача 4 (MyMap<int, Money> - умножение на максимум)
5. Задача 5 (MyMap<int, Money> - добавление минимума)
0. Выход

Ваш выбор: 1

ЗАДАЧА 1

Контейнер: multimap

Тип: float

Введите 5 пар (ключ int, значение float):

Пара 1 - ключ: 5

значение: 1

Пара 2 - ключ: 10.5

значение: Пара 3 - ключ: 2

значение: 20.3

Пара 4 - ключ: 3

значение: 5.2

Пара 5 - ключ: 4

значение: 30.1

Исходный multimap:

Ключ: 2, Значение: 20.3

Ключ: 3, Значение: 5.2

Ключ: 4, Значение: 30.1

Ключ: 5, Значение: 1

Ключ: 10, Значение: 0.5

Минимальный элемент: 0.5

ЗАДАЧА 2
Контейнер: multimap
Тип: Money

Введите 3 пары (ключ int, значение Money):

Пара 1
Ключ: 1
Значение (Money):
Введите рубли: 2
Введите копейки: 30

Пара 2
Ключ: 2
Значение (Money):
Введите рубли: 12
Введите копейки: 30

Пара 3
Ключ: 3
Значение (Money):
Введите рубли: 40
Введите копейки: 40

Исходный multimap:
Ключ: 1, Значение: 2,30
Ключ: 2, Значение: 12,30
Ключ: 3, Значение: 40,40

Среднее арифметическое: 18.3333

Элементы больше среднего:
Удален: ключ 3, значение 40,40

Результат после удаления:
Ключ: 1, Значение: 2,30
Ключ: 2, Значение: 12,30

```
ЗАДАЧА 3
Параметризированный класс: MyMap
Тип: int, Money

Введите 3 пары (ключ int, значение Money):

Пара 1
Ключ: 1
Значение (Money):
Введите рубли: 80
Введите копейки: 50

Пара 2
Ключ: 2
Значение (Money):
Введите рубли: 50
Введите копейки: 80

Пара 3
Ключ: 3
Значение (Money):
Введите рубли: 40
Введите копейки: 50

Исходный контейнер:
Ключ: 1, Значение: 80,50
Ключ: 2, Значение: 50,80
Ключ: 3, Значение: 40,50

Результат после удаления элементов больше среднего:
Ключ: 2, Значение: 50,80
Ключ: 3, Значение: 40,50
```

```
ЗАДАЧА 4 (Дополнительно)
МуМар с операцией умножения на максимальный элемент

Введите 3 пары (ключ int, значение Money):

Пара 1
Ключ: 1
Значение (Money):
Введите рубли: 30
Введите копейки: 9

Пара 2
Ключ: 2
Значение (Money):
Введите рубли: 20
Введите копейки: 30

Пара 3
Ключ: 3
Значение (Money):
Введите рубли: 23
Введите копейки: 30

Исходный контейнер:
Ключ: 1, Значение: 30,09
Ключ: 2, Значение: 20,30
Ключ: 3, Значение: 23,30

Максимальный элемент: 30,09

Результат после умножения каждого элемента на максимальный:
Ключ: 1, Значение: 30,09
Ключ: 2, Значение: 610,82
Ключ: 3, Значение: 701,09
```

```

ЗАДАЧА 5 (Дополнительно)
МультиМат с операцией добавления минимального элемента

Введите 3 пары (ключ int, значение Money):

Пара 1
Ключ: 1
Значение (Money):
Введите рубли: 20
Введите копейки: 29

Пара 2
Ключ: 30
Значение (Money):
Введите рубли: 30
Введите копейки: 23

Пара 3
Ключ: 3
Значение (Money):
Введите рубли: 40
Введите копейки: 35

Исходный контейнер:
Ключ: 1, Значение: 20,29
Ключ: 3, Значение: 40,35
Ключ: 30, Значение: 30,23

Минимальный элемент: 20,29
Введите ключ для вставки минимального элемента: 29

Результат после добавления минимального элемента:
Ключ: 1, Значение: 20,29
Ключ: 3, Значение: 40,35
Ключ: 29, Значение: 20,29
Ключ: 30, Значение: 30,23

```

Ответы на вопросы

Ниже ответы на вопросы по ассоциативным контейнерам STL в C++. Сухо, по факту, без выделений.

1. Ассоциативный контейнер — это контейнер, в котором доступ к элементам осуществляется по ключу, а не по позиции. Каждый элемент имеет ключ и значение (или только ключ).
2. Ассоциативные контейнеры STL: map, multimap, set, multiset, unordered_map, unordered_multimap, unordered_set, unordered_multiset.
3. Доступ к элементам ассоциативного контейнера осуществляется по ключу с помощью оператора [] (для map), метода at(), метода find(), а также с помощью итераторов.
4. Примеры методов: insert, erase, find, count, lower_bound, upper_bound, equal_range, begin, end, size, empty, clear, at, operator[].
5. Создание контейнера map:

```
map<int, string> m1;
```

```
map<string, int> m2 = {{ "a",1}, {"b",2}};
```

```
map<int, string, greater<int>> m3;
```

6. По умолчанию элементы в map упорядочены по возрастанию ключа (компаратор `less<Key>`). Для изменения порядка на обратный используется компаратор `greater<Key>`:

```
map<int, string, greater<int>> m;
```

7. Операции для map: вставка (`insert`, `operator[]`), удаление (`erase`), поиск (`find`, `at`, `operator[]`), проверка наличия (`count`), доступ к началу и концу (`begin`, `end`), размер (`size`), проверка пустоты (`empty`), очистка (`clear`).

8. Функция добавления элементов с помощью `make_pair`:

```
void addToMap(map<int, string>& m, int key, const string& value)
{
    m.insert(make_pair(key, value));
}
```

9. Функция добавления элементов с помощью операции `[]`:

```
void addToMap(map<int, string>& m, int key, const string& value)
{
    m[key] = value;
}
```

10. Функция печати map с помощью итератора:

```
void printMap(const map<int, string>& m)
{
    for (auto it = m.begin(); it != m.end(); ++it)
    {
        cout << it->first << ": " << it->second << endl;
    }
}
```

11. Функция печати map с помощью операции [] не используется, так как operator[] не работает с константным map и создаёт отсутствующие элементы. Правильный способ — через итераторы или range-based for.

12. Отличие map от multimap: map хранит уникальные ключи (каждый ключ только один раз), multimap допускает повторяющиеся ключи. В map есть operator[] и at, в multimap их нет.

13. Контейнер set — это ассоциативный контейнер, хранящий уникальные отсортированные значения. В set нет явного разделения на ключ и значение — само значение является ключом.

14. Отличия map от set: map хранит пары ключ-значение, set хранит только значения. К элементам map доступ по ключу даёт значение, в set доступ по значению даёт само значение. В map есть operator[] и at, в set их нет.

15. Создание контейнера set:

```
set<int> s1;  
set<string> s2 = {"a", "b", "c"};  
set<int, greater<int>> s3;
```

16. По умолчанию элементы в set упорядочены по возрастанию (less<Key>). Для изменения порядка на обратный используется компаратор greater<Key>:

```
set<int, greater<int>> s;
```

17. Операции для set: вставка (insert), удаление (erase), поиск (find), проверка наличия (count), получение диапазонов (lower_bound, upper_bound), доступ к началу и концу (begin, end), размер (size), проверка пустоты (empty), очистка (clear).

18. Функция добавления элементов в set:

```
void addToSet(set<int>& s, int value)  
{
```

```
s.insert(value);  
}
```

19. Функция печати set:

```
void printSet(const set<int>& s)  
{  
    for (auto it = s.begin(); it != s.end(); ++it)  
    {  
        cout << *it << " ";  
    }  
}
```

20. Отличие set от multiset: set хранит уникальные значения, multiset допускает повторяющиеся значения. Методы count в set возвращает 0 или 1, в multiset — любое количество.